

46-926, Spring 2019: Homework #5

Due **5PM**, February 25, 2019.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

Remark: This homework was posted a bit late because I was traveling, so I haven't made it quite as long as some of the others. However, please make sure that you can install the `pdpbox` package early, in case you have problems.

Correction (2/22/19 at 4:15pm): Changed the instructions in problem 3 to refer to the random forest in problem 2b, not 1b.

1. *Decision trees, marketing, and imbalanced data.* For this problem and the next couple, we will be working with our favorite marketing data sample, `marketing.csv`, just so we already know what to expect.

You should do this problem in R using the `rpart` package. The decision tree functions in python are a bit too limited for our purposes here.

Note: In class, we introduced the `tree` package for fitting decision trees in R. For this homework we will use the competing package `rpart`, partly because it is often more stable and is worth knowing, and partly because `tree` behaves particularly poorly on this data set. Most of what you need is below, but I encourage you to read the help file for each new function as you go.

Note: For plotting the ROC curves, I ask you to compare to the logistic model. You can pass in the scores directly from python if you have `rpy2` working. Otherwise you can write them out to csv and read them in, or just refit the logistic model from within R using the `glm` function.

- (a) Fit a decision tree using the `rpart` package with no pruning. To do this, use the `rpart` function from that package. The syntax is almost identical to the `tree` package, but it's worth looking at the help file.

Try to plot the resulting tree using the `plot` command. What goes wrong? What property of this data set do you think makes it hard to construct a good tree?

Note: To understand this outcome, it is helpful to know that, by default, the `rpart` function won't make a split (even before pruning) if it looks particularly useless. You can see more in the help file for `rpart.control`, if you feel like it.

- (b) To help deal with the imbalance of the data set, we will assign higher *weight* to the rare 'yes' class. This will help the tree look for splits that favor this rare class.

Define a vector `wgts` of weights, so that `wgts` is 10 when `y=='yes'` and `wgts` is 1 when `y=='no'`. Rerun `rpart`, but set `weights=wgts` in your call.

Now plot the resulting tree, using the `plot` and `text` functions.

What is the misclassification rate and how does it compare to the base rate and to logistic regression?

What is the rate of success in the top 1000 highest rated individuals? (Use `predict` with `type='prob'` and look at the second column to get scores. You may be bothered by the fact that there are many ties, but just ignore it for now.)

- (c) Prune that tree. You can plot the cross-validation curve with `plotcp`. From the cross-validation picture, pick a reasonable value of `cp` (the tuning parameter you are cross-validating) and explain why you picked it. Use the `prune` function with that value of `cp` to obtain a new, pruned tree.

Again, plot the resulting tree and assess the misclassification rate and the top 1000 set.

- (d) Plot ROC curves for your two trees, along with logistic regression. How do they compare? Why are the tree curves extremely simple (piecewise linear with only a few segments)?

Hint:

You can plot ROC curves in R with the `roc` function in the `pROC` package. The format is

```
roc(true_categories, scores, plot=TRUE, col='red')
```

where `true_categories` is the outcome you are predicting, `scores` are the classifier scores, and `red` is whatever color you want.

To plot multiple ROC curves, you can use the `add=TRUE` option when making

repeated calls to this function.

2. *Random forests, marketing, and imbalanced data* Continuing on with the same data set, let's see what random forests have to add over the decision tree.

Unlike the last problem, let's switch back to python. For comparisons in this problem, you'll want the scores for logistic regression and GAM from previous homeworks.

Note: You will need to transform your categorical variables to dummy coding first. Use the appropriate function in pandas. I recommend doing this before splitting your data, so you can do it once. I also recommend leaving all the variables in (the default), rather than dropping one to make it linearly independent.

- (a) Let's see if you can do better than in previous homeworks. Fit a random forest to your training data using the `python RandomForestClassifier` function. Use 500 trees and set `oob_score=True`. Leave the rest of the parameters at their defaults for now.

- i. What is your misclassification rate? Compare to the base rate and the misclassification rates for your other classifiers.

How does your misclassification rate compare to your OOB error for this forest?

- ii. Plot an ROC curve overlaying all three classifiers and compare their performance. Include a legend. Also compare the AUC of your classifier to those of the other classifiers.

- iii. How good is your top 1000 set for this classifier, and how does it compare to the other classifiers?

- (b) Let's try a different node size. This is one of the parameters that can actually have an impact on random forest fits, especially in imbalanced settings. Repeat the previous part, but set the `min_samples_leaf` parameter to 10.

- (c) Let's see if we can improve further! The naive random forest can suffer when samples are highly imbalanced, since splits will tend not to favor the rare class, and since leaf nodes will usually be dominated by the common class. A variety of adjustments have been proposed to help correct for this. One of the simplest approaches is to shift the weight given to each class (kind of like the importance weights in boosting, except only based on outcome class).

You can implement this in python by setting the `class_weight` parameter to `balanced`. This adds weight proportionally to the rarer class.

Fit this balanced version of the random forest. Again, compare misclassification, AUC, and top 1000 set. Create a new ROC plot showing all four classifiers. How do the two random forest classifiers compare?

Do you notice something strange about your misclassification rate here? Why do you think that is?

3. Now that we have a somewhat decent classifier, let's try to get a peek at how it works.
 - (a) Create a variable importance plot for the random forest model from problem 2b. What are the two most important variables to your classifier?
 - (b) For the two most important variables, create univariate partial dependence plots. How do your predictions seem to depend on these variables, roughly?

Hints:

The random forest in sklearn does not have its own partial dependence plot function. However, the package `pdpbox` can be used to create a partial dependence plot for a given function. Basic usage is as follows:

```
from pdpbox import pdp
pdp_obj = pdp.pdp_isolate(model=your_model, dataset=data,
model_features=your_features, feature=target_feature)
fig, axes = pdp.pdp_plot(pdp_obj, 'Feature Name')
```

where `your_model` is the fitted model, `data` is a data set, `model_features` is a list of all the features you fed your classifier (can get this with `X.columns` for an `X` matrix), and `feature` is the feature you want to examine.

You can see more examples at https://github.com/SauceCat/PDPbox/blob/master/tutorials/pdpbox_binary_classification.ipynb

The package information and installation information can be found at <https://pdpbox.readthedocs.io/en/latest/>. You should be able to install it through your python package manager.

4. *Bad validation.* Here we demonstrate one way that validation of ML methods can go wrong. This ties back to some earlier discussions about cross-validation, and also leads into some practical discussions we will have in Lecture 7.

This problem will walk you through a common (but wrong) form of validation. *Don't do this in real life.*

The file `bad_valid.py` contains starter code for this problem. You will use the function `get_data` to generate your data sets.

- (a) Generate a data set ($n = 100$) using the `get_data` function. We will be using this data set for both training and testing by using a validation split.

First, you decide to investigate how useful your variables are for understanding y . Compute a vector of correlations between each column of X and the outcome vector y .

You can compute the correlation between y and a single column of X using the `pearsonr` function from `scipy.stats`. I recommend using `np.apply_along_axis` to compute correlations with all the columns at once.

Plot a histogram of the correlations between y and each of the columns of X in your training set.

You should notice a nice bell-shaped curve of correlations around zero. (You might expect this in real life, since you know that you are just analyzing noise.)

- (b) Suppose you decide that you have too many variables. Since you've noticed that most of the variables are worthless, you decide to discard those worthless variables before running your fancy ML methods.

Figure out which columns of X have absolute value correlation with y bigger than 0.2. Make a new data frame that only has those highly correlated columns; call it `X_good`. (You will probably want to keep track of the indices of these columns, since you'll need to make this matrix again later.)

- (c) Split your `X_good` data set into the first $0.8*n$ points (training set) and the last $0.2*n$ points (test set). Think of this like back testing: you're going to train on the older data and predict on the more recent data.

Train a lasso model on the first data set, using the `LassoCV` function from `sklearn.linear_models`, with cross-validation to pick your regularization parameter.

What is the cross-validation estimate for your model's mean squared error? Use your model to predict on the test portion of the `X_good` data set. What is your test mean squared error?

At this point, everything should be looking pretty good!

Hints:

To get the cross-validated MSE, note that the `mse_path_` object contains all your cross-validated MSEs for each regularization parameter value and every fold separately. Taking averages across the rows (folds) will give you the CV-MSE at each regularization parameter value. The minimum of these is your cross-validated MSE for the selected model.

- (d) Generate another data set of size 100 from `get_data` and assess your model on this fresh data. You will have to first extract the same columns that you extracted when you formed `X_good`, and then use your lasso model to predict with those subset columns. What is your mean squared error on this fresh data? Explain why this number is quite different from your cross-validated error and your test error on the previously held-out data set?